

# 实验1 现代C++编程初体验

---

## 说明

### 一、实验目的

### 二、实验准备

### 三、实验内容

#### 1. 实验任务1

#### 2. 实验任务2

#### 3. 实验任务3

#### 4. 实验任务4

#### 5. 实验任务5

#### 6. 实验任务6

#### 7. 实验任务7

### 四、实验结论

#### 1. 实验任务1

#### 2. 实验任务2

#### 3. 实验任务3

#### 4. 实验任务4

#### 5. 实验任务5

#### 6. 实验任务6

#### 7. 实验任务7

### 五、实验总结(选)

### 六、实验文档提交要求

### 七、博客园编辑器使用补充说明

## 说明

---

### 总体说明

现代C++融合了结构化、面向对象、泛型与函数式多种编程范式：早年从Simula汲取面向对象精髓，之后博采众长，成为高效驾驭硬件、管理复杂性的工业级利器。其应用从贝尔实验室扩展至操作系统、金融、医疗、航天、游戏、机器学习等诸多领域。

课堂所接触的标准库功能，仅是管中窥豹。本次实验作为入门，期待你在"动手实践 → 发现问题 → 深入探索"的循环中，将零散的知识点组装起来，让理论与实践双向驱动。其中：

- 任务1-3为验证性实验，以体验为主；
- 任务4-7为应用编程练习，旨在训练综合运用知识解决基础问题的能力。

### 实验须知

#### • 阅读文档

请仔细阅读文档，明确各项任务的具体要求、提交方式与截止时间。

#### • 动手实践

如果你不熟悉C++语法，请手动输入代码，通过实践熟悉语法规则。

亲自动手敲一遍代码与从文档中直接复制运行，练习效果完全不同。

#### • 提交规范

- 一次实验的所有任务写在一篇博客中

- 发布博客后，还需要在作业系统中提交链接
- 晚交的实验，按课程规则处理（分数折半）

## 一、实验目的

1. **体验标准库**：通过使用 `string`, `vector`, `algorithm` 等组件，体验现代C++标准库编程的便捷与高效。
2. **组合运用**：灵活组合使用现代C++基础语言特性（数据表示、分支、循环、函数）和标准库，编程解决基础问题。
3. **培养编码素养**：在编码过程中关注代码表达，提升代码的安全性、可读性和可维护性。
4. **理解模板**：通过使用函数模板，初步理解泛型编程的思想和优势。

## 二、实验准备

- 系统浏览教材以下章节，对现代c++基础语言特性和用法构建全局认知
  - 第2章 C++语言简单程序设计
  - 第3章 函数
  - 第9章 9.1.1节 函数模板
- 通过以下网站（或类似工具网站），学习现代C++标准库中标准库组件（如 `string`, `vector`）和算法工具（如 `sort`, `reverse`）的基本用法。

<https://en.cppreference.com/w/>

<https://www.runoob.com/cplusplus/cpp-tutorial.html>

## 三、实验内容

### 1. 实验任务1

验证性实验。

在C++编码环境中，输入、运行并观察以下代码，结合运行结果和注释，体验使用C++标准库进行编程的便捷性。从面向对象编程范式的角度，体会**封装**与**基于接口编程**的意义。

task1.cpp

```
1 // 现代C++标准库、算法库体验
2 // 本例用到以下内容：
3 // 1. 字符串string，动态数组容器类vector、迭代器
4 // 2. 算法库：反转元素次序、旋转元素
5 // 3. 函数模板、const引用作为形参
6
7 #include <iostream>
8 #include <string>
9 #include <vector>
10 #include <algorithm>
```

```
11
12 // 模板函数声明
13 template<typename T>
14 void output(const T &c);
15
16 void test1();
17 void test2();
18 void test3();
19
20 int main() {
21     std::cout << "测试1: \n";
22     test1();
23
24     std::cout << "\n测试2: \n";
25     test2();
26
27     std::cout << "\n测试3: \n";
28     test3();
29 }
30
31 // 输出容器对象c中的元素
32 template <typename T>
33 void output(const T &c) {
34     for(auto &i : c)
35         std::cout << i << ' ';
36     std::cout << '\n';
37 }
38
39 // 测试1: 组合使用算法库、迭代器、string反转字符串
40 void test1() {
41     using namespace std;
42
43     string s0{"0123456789"};
44     cout << "s0 = " << s0 << endl;
45
46     string s1(s0);
47     // 反转s1自身
48     reverse(s1.begin(), s1.end());
49     cout << "s1 = " << s1 << endl;
50
51     string s2(s0.size(), ' ');
52     // 将s0反转后结果拷贝到s2, s0自身不变
53     reverse_copy(s0.begin(), s0.end(), s2.begin());
54     cout << "s2 = " << s2 << endl;
55 }
56
57 // 测试2: 组合使用算法库、迭代器、vector反转动态数组对象vector内数据
58 void test2() {
59     using namespace std;
60
61     vector<int> v0{2, 0, 4, 9};
62     cout << "v0: "; output(v0);
63 }
```

```

64     vector<int> v1{v0};
65     reverse(v1.begin(), v1.end());
66     cout << "v1: "; output(v1);
67
68     vector<int> v2{v0};
69     reverse_copy(v0.begin(), v0.end(), v2.begin());
70     cout << "v2: "; output(v2);
71 }
72
73 // 测试3: 组合使用算法库、迭代器、vector实现元素旋转移位
74 void test3() {
75     using namespace std;
76
77     vector<int> v0{0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
78     cout << "v0: "; output(v0);
79
80     vector<int> v1{v0};
81     // 将[v1.begin(), v1.end())区间内元素循环左移1位
82     rotate(v1.begin(), v1.begin()+1, v1.end());
83     cout << "v1: "; output(v1);
84
85     vector<int> v2{v0};
86     // 将[v1.begin(), v1.end())区间内元素循环左移2位
87     rotate(v2.begin(), v2.begin()+2, v2.end());
88     cout << "v2: "; output(v2);
89
90     vector<int> v3{v0};
91     // 将[v1.begin(), v1.end())区间内元素循环右移1位
92     rotate(v3.begin(), v3.end()-1, v3.end());
93     cout << "v3: "; output(v3);
94
95     vector<int> v4{v0};
96     // 将[v1.begin(), v1.end())区间内元素循环右移2位
97     rotate(v4.begin(), v4.end()-2, v4.end());
98     cout << "v4: "; output(v4);
99 }

```

程序正确录入、编译后，运行测试结果：

```
测试1:
s0 = 0123456789
s1 = 9876543210
s2 = 9876543210

测试2:
v0: 2 0 4 9
v1: 9 4 0 2
v2: 9 4 0 2

测试3:
v0: 0 1 2 3 4 5 6 7 8 9
v1: 1 2 3 4 5 6 7 8 9 0
v2: 2 3 4 5 6 7 8 9 0 1
v3: 9 0 1 2 3 4 5 6 7 8
v4: 8 9 0 1 2 3 4 5 6 7
```

观察与思考：

- `reverse` 和 `reverse_copy` 有什么区别？
- `rotate` 算法是如何改变元素顺序的？它的三个参数分别代表什么？

## 2. 实验任务2

验证性实验。

在C++编码环境中，输入、运行并观察以下代码，体验使用C++标准库高效完成数据赋值、排序和基本统计。

task2.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <numeric>
5  #include <iomanip>
6  #include <cstdlib>
7  #include <ctime>
8
9  // 模板函数声明
10 template<typename T>
11 void output(const T &c);
12
13 int generate_random_number();
14 void test1();
15 void test2();
16
17 int main() {
18     std::srand(std::time(0)); // 添加随机种子
19     std::cout << "测试1: \n";
20     test1();
21 }
```

```
22     std::cout << "\n测试2: \n";
23     test2();
24 }
25
26 // 输出容器对象c中的元素
27 template <typename T>
28 void output(const T &c) {
29     for(auto &i: c)
30         std::cout << i << ' ';
31     std::cout << '\n';
32 }
33
34 // 返回[0, 100]区间内的一个随机整数
35 int generate_random_number() {
36     return std::rand() % 101;
37 }
38
39 // 测试1: 对容器类对象指定迭代器区间赋值、排序
40 void test1() {
41     using namespace std;
42
43     vector<int> v0(10); // 创建一个动态数组对象v0, 对象大小为10
44     generate(v0.begin(), v0.end(), generate_random_number); // 生成随机数填充v0
45     cout << "v0: "; output(v0);
46
47     vector<int> v1{v0};
48     sort(v1.begin(), v1.end()); // 对整个vector排序
49     cout << "v1: "; output(v1);
50
51     vector<int> v2{v0};
52     sort(v2.begin()+1, v2.end()-1); // 只对中间部分排序, 不包含首尾元素
53     cout << "v2: "; output(v2);
54 }
55
56 // 测试2: 对容器类对象指定迭代器区间赋值、计算最大值/最小值/均值
57 void test2() {
58     using namespace std;
59
60     vector<int> v0(10);
61     generate(v0.begin(), v0.end(), generate_random_number);
62     cout << "v0: "; output(v0);
63
64     // 求最大值和最小值
65     auto min_iter = min_element(v0.begin(), v0.end());
66     auto max_iter = max_element(v0.begin(), v0.end());
67     cout << "最小值: " << *min_iter << endl;
68     cout << "最大值: " << *max_iter << endl;
69
70     // 同时求最大值和最小值
71     auto ans = minmax_element(v0.begin(), v0.end());
72     cout << "最小值: " << *(ans.first) << endl;
73     cout << "最大值: " << *(ans.second) << endl;
74 }
```

```

75 // 求平均值
76 double avg1 = accumulate(v0.begin(), v0.end(), 0.0) / v0.size();
77 cout << "均值: " << fixed << setprecision(2) << avg1 << endl;
78
79 sort(v0.begin(), v0.end());
80 double avg2 = accumulate(v0.begin()+1, v0.end()-1, 0.0) / (v0.size()-2);
81 cout << "去掉最大值、最小值之后, 均值: " << avg2 << endl;
82 }

```

正确正确录入、编译后, 运行测试结果:

```

测试1:
v0: 51 3 81 64 75 89 84 5 90 69
v1: 3 5 51 64 69 75 81 84 89 90
v2: 51 3 5 64 75 81 84 89 90 69

测试2:
v0: 90 13 37 98 68 33 93 46 68 9
最小值: 9
最大值: 98
最小值: 9
最大值: 98
均值: 55.50
去掉最大值、最小值之后, 均值: 56.00

```

观察与思考:

- `generate` 算法的作用是什么?
- `minmax_element` 和分别调用 `min_element`、`max_element` 相比, 有什么优势?
- 把代码中函数 `generate_random_number` 的声明 (line13) 和定义 (line35-37) 注释起来, 把两处调用改成如下写法, 观察效果是否等同? 查阅c++中lambda表达式用法。

```

1 // test1() 模块和 test2() 模块:
2 generate(v0.begin(), v0.end(), generate_random_number);
3 ----> 改成
4 generate(v0.begin(), v0.end(), []() { return std::rand() % 101; });

```

### 3. 实验任务3

验证性实验。

在C++编码环境中, 输入、运行并观察以下代码, 体验使用C++标准库高效完成字符串变换操作。

task3.cpp

```

1 #include <iostream>
2 #include <string>
3 #include <algorithm>
4 #include <cctype>
5
6 unsigned char func(unsigned char c);

```

```

7 void test1();
8 void test2();
9
10 int main() {
11     std::cout << "测试1: 字符串大小写转换\n";
12     test1();
13
14     std::cout << "\n测试2: 字符变换\n";
15     test2();
16 }
17
18 unsigned char func(unsigned char c) {
19     if(c == 'z')
20         return 'a';
21
22     if(c == 'Z')
23         return 'A';
24
25     if(std::isalpha(c))
26         return static_cast<unsigned char>(c+1);
27
28     return c;
29 }
30
31 void test1() {
32     std::string s1{"Hello World 2049!"};
33     std::cout << "s1 = " << s1 << '\n';
34
35     std::string s2;
36     for(auto c: s1)
37         s2 += std::tolower(c);
38     std::cout << "s2 = " << s2 << '\n';
39
40     std::string s3;
41     for(auto c: s1)
42         s3 += std::toupper(c);
43     std::cout << "s3 = " << s3 << '\n';
44 }
45
46 void test2() {
47     std::string s1{"I love cosmos!"};
48     std::cout << "s1 = " << s1 << '\n';
49
50     std::string s2(s1.size(), ' ');
51     std::transform(s1.begin(), s1.end(),
52                   s2.begin(),
53                   func);
54     std::cout << "s2 = " << s2 << '\n';
55 }

```

正确正确录入、编译后，运行测试结果：



```
测试1: 字符串大小写转换
s1 = Hello World 2049!
s2 = hello world 2049!
s3 = HELLO WORLD 2049!

测试2: 字符变换
s1 = I love cosmos!
s2 = J mpwf dptnpt!
```

观察与思考:

- 自定义函数 `func` 功能是什么?
- `tolower` 和 `toupper` 功能分别是什么?
- `transform` 的4个参数意义分别是什么? 如果把第3个参数 `s2.begin()` 改成 `s1.begin()`, 有何区别?

## 4. 实验任务4

编写函数, 实现判断回文串。是返回 `true`, 否则, 返回 `false`。

在 `main` 函数中调用, 输入字符串, 调用其判断, 输出结果。要求支持多组输入。

- 严格区分大小写: `bool is_palindrome(const std::string &s)`
- 不区分大小写: `bool is_palindrome_ignore_case(const std::string &s);`

Tips\*:

参考验证性实验任务3, 思考如何运用字符串变换来实现「不区分大小写回文」判断。

待补足代码task3.cpp

```
1  #include <iostream>
2  #include <string>
3  #include <algorithm>
4
5  bool is_palindrome(const std::string &s);
6  bool is_palindrome_ignore_case(const std::string &s);
7
8  int main() {
9      using namespace std;
10     string s;
11
12     // 多组输入, 直到按下Ctrl+Z结束测试
13     while(cin >> s) {
14         cout << boolalpha
15              << "区分大小写: " << is_palindrome(s) << "\n"
16              << "不区分大小写: " << is_palindrome_ignore_case(s) << "\n\n";
17     }
18 }
19
20
21 // 函数is_palindrome定义
22 // 待补足
23 // xxx
24
```

```

25
26 // 函数is_palindrome_ignore_case定义
27 // 待补足
28 // xxx

```

程序正确补足、编译后，预期测试结果如下：

```

Ada
区分大小写: false
不区分大小写: true

Wow
区分大小写: false
不区分大小写: true

nice
区分大小写: false
不区分大小写: false

Level
区分大小写: false
不区分大小写: true

x
区分大小写: true
不区分大小写: true

^Z

```

**观察与思考：**

使用 `cin >> s` 输入时，输入的字符串中不能包含空格。如果希望测试字符串包含空格（如 `hello oop`），代码应如何调整？

## 5. 实验任务5

编写程序实现进制转换。具体要求如下：

- 编写函数 `std::string dec2n(x, n)` 实现把一个十进制数 `x` 转换成 `n` 进制，结果以字符串形式返回。如果第2个参数没有指定，默认转换成二进制。（本次实验，限定参数 `n` 取值区间为 `[2, 36]`，限定参数 `x >= 0`）

当 `n = 17`，即17进制时，基数是0-9, A-G

当 `n = 18`，即18进制时，基数是0-9, A-H

依次类推：

当 `n = 36`，即36进制时，基数是0-9, A-Z （即：10 -> A, 11->B, 12->C, ..., 35->Z）

- 在 `main` 函数中，输入十进制整数，调用函数 `dec2n` 得到转换成指定进制的字符串，输出。要求支持多组输入。

待补足的代码：

task5.cpp

```

1 #include <iostream>
2 #include <string>
3 #include <algorithm>

```

```

4
5 std::string dec2n(int x, int n = 2);
6
7 int main() {
8     int x;
9     while(std::cin >> x) {
10         std::cout << "十进制: " << x << '\n'
11                 << "二进制: " << dec2n(x) << '\n'
12                 << "八进制: " << dec2n(x, 8) << '\n'
13                 << "十二进制: " << dec2n(x, 12) << '\n'
14                 << "十六进制: " << dec2n(x, 16) << '\n'
15                 << "三十二进制: " << dec2n(x, 32) << "\n\n";
16     }
17 }
18
19 // 函数dec2n定义
20 // 待补足
21 // xxx

```

程序正确补足、编译后，预期测试结果如下：

```

0
十进制: 0
二进制: 0
八进制: 0
十二进制: 0
十六进制: 0
三十二进制: 0

42
十进制: 42
二进制: 101010
八进制: 52
十二进制: 36
十六进制: 2A
三十二进制: 1A

55
十进制: 55
二进制: 110111
八进制: 67
十二进制: 47
十六进制: 37
三十二进制: 1N

^Z

```

## 6. 实验任务6

编写程序，在屏幕上打印字母密文对照表。

程序正确编写后，预期输出如下：

|    |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|    | a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v | w | x | y | z |
| 1  | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A |
| 2  | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B |
| 3  | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C |
| 4  | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D |
| 5  | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E |
| 6  | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F |
| 7  | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G |
| 8  | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H |
| 9  | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I |
| 10 | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J |
| 11 | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K |
| 12 | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L |
| 13 | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M |
| 14 | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N |
| 15 | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O |
| 16 | Q | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P |
| 17 | R | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q |
| 18 | S | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R |
| 19 | T | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S |
| 20 | U | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T |
| 21 | V | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U |
| 22 | W | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V |
| 23 | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W |
| 24 | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X |
| 25 | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y |
| 26 | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z |

## 7. 实验任务7

编写一个程序，实现自动生成小学生算术运算题目并自动评测。具体要求如下：

- 程序运行后，自动生成10道10以内的算术运算题。
  - 运算式（包括=）在程序运行时自动生成。  
运算式=右侧的答案由用户输入
  - 算术运算包括：加、减、乘、除。生成题目时，运算随机。
  - 两个操作数限制在[1, 10]区间内的整数。
- 题目是减法运算时，要求第一个操作数>=第二个操作数。
- 题目是除法运算时，要求第一个操作数能整除第二个操作数整除。
- 用户完成10道测试后，屏幕给出正确率，要求以百分号形式输出，保留小数点后两位。
- 多次运行程序时，每次生成的题目与上次不同。（Tips: 使用 `srand(time(0))` 作为随机种子）

程序正确编写后，预期测试效果如下：（截图中给了两次运行测试效果）

|             |             |
|-------------|-------------|
| 8 / 2 = 4   | 8 / 4 = 2   |
| 6 / 2 = 3   | 6 + 8 = 14  |
| 3 + 10 = 13 | 2 - 2 = 0   |
| 5 / 5 = 1   | 3 - 1 = 2   |
| 9 / 1 = 9   | 8 * 7 = 15  |
| 2 + 6 = 8   | 4 + 6 = 10  |
| 1 * 8 = 8   | 9 - 2 = 6   |
| 4 * 10 = 40 | 4 + 6 = 10  |
| 6 / 3 = 3   | 8 / 2 = 4   |
| 6 - 5 = 1   | 6 * 2 = 12  |
| 正确率: 90.00% | 正确率: 80.00% |

## 四、实验结论

### 1. 实验任务1

此部分书写内容:

- 给出程序源代码task1.cpp, 及, 运行测试截图
- 回答问题
  - 问题1: `reverse` 和 `reverse_copy` 有什么区别?
  - 问题2: `rotate` 算法是如何改变元素顺序的? 它的三个参数分别代表什么?

### 2. 实验任务2

此部分书写内容:

- 给出程序源代码task2.cpp, 及, 运行测试截图
- 回答问题
  - 问题1:** `generate` 算法的作用是什么?
  - 问题2:** `minmax_element` 和分别调用 `min_element`、`max_element` 相比, 有什么优势?
  - 问题3:** 查询 `generate` 第3个参数 `[]() {return std::rand()%101;}` 用法, 与使用自定义函数 `generate_random_number` 相比, lambda表达式适用场景是什么?

```
1 | generate(v0.begin(), v0.end(), []() {return std::rand()%101;});
```

### 3. 实验任务3

此部分书写内容:

- 给出程序源代码task3.cpp, 及, 运行测试截图
- 回答问题
  - 问题1:** 自定义函数 `func` 功能是什么?
  - 问题2:** `tolower` 和 `toupper` 功能分别是什么?
  - 问题3:** `transform` 的4个参数意义分别是什么? 如果把第3个参数 `s2.begin()` 改成 `s1.begin()`, 有何区

别？

## 4. 实验任务4

此部分书写内容：

- 给出程序源代码task4.cpp，及，运行测试截图
- 回答问题

**问题：**使用 `cin >> s` 输入时，输入的字符串中不能包含空格。如果希望测试字符串包含空格（如 `hello oop`），代码应如何调整？

## 5. 实验任务5

此部分书写内容：

- 给出实现题目要求的源代码task5.cpp，及，运行测试截图

## 6. 实验任务6

此部分书写内容：

- 给出实现题目要求的源代码task6.cpp，及，运行测试截图

## 7. 实验任务7

此部分书写内容：

- 给出实现题目要求的源代码task7.cpp，及，运行测试截图

## 五、实验总结(选)

此部分书写内容：（可以包括但不限于以下内容）

- 提炼本次实验的核心收获与思考
- 记录新发现、真实感受与待解疑问
- 任意想补充的反馈或分享

## 六、实验文档提交要求

- **提交形式**

请将实验文档以**在线技术博客**形式提交，发布后请在**相应实验作业**页面提交链接。

博客开通与提交方法详见：「[博客园博客开通及发布步骤\\_2024级\\_OOP.pdf](#)」

- **撰写内容**

**必写：**「四、实验结论」

**选写：**「五、实验总结」

**要求：**内容完整，表述简练，逻辑清晰，图文整洁

- 截止时间

通常为实验课后一周内，具体**截止时间**以博客园发布的作业通知为准。（**提交时间**以博客园文章页面显示的日期为准）

截止后，欢迎通过[教学班级博客主页](#)相互评阅。

代码正确、风格规范、图文清晰、总结到位的博客，欢迎点赞推荐，一起进步！

## 七、博客园编辑器使用补充说明

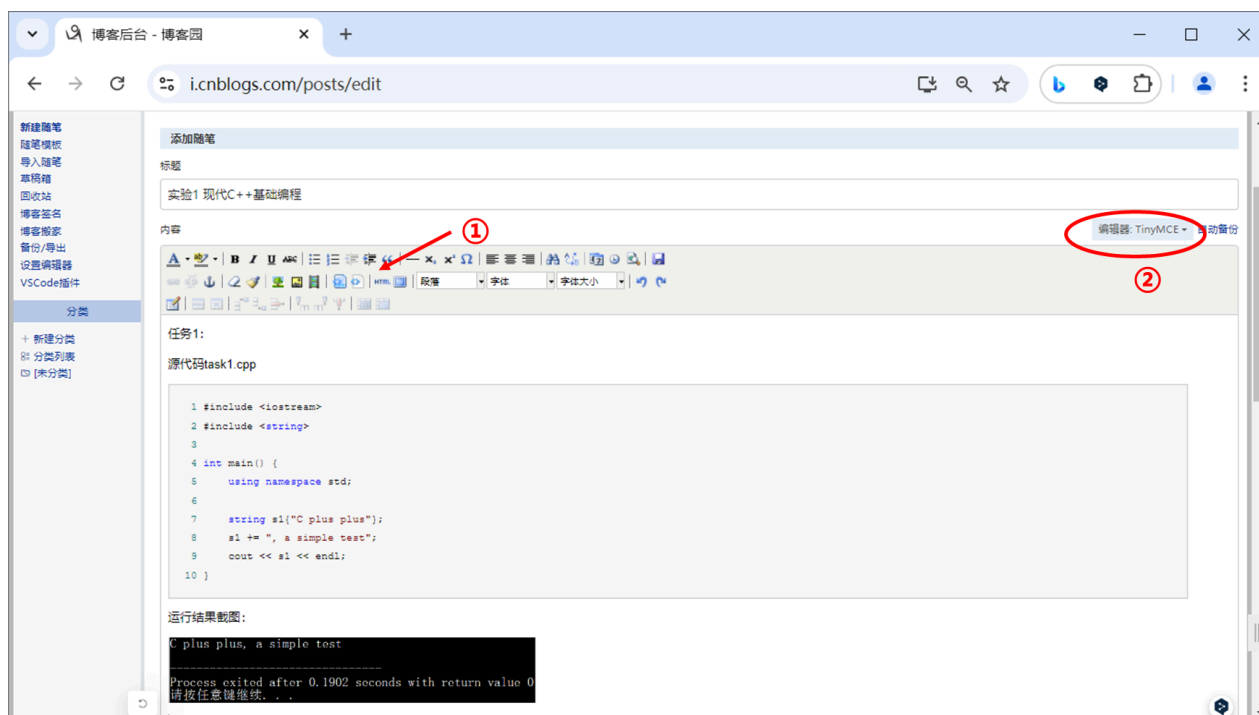
针对博客园平台使用的默认编辑器，插入代码和图片后，文字难以插入编辑的状况：

- 解决方案1

在插入代码和图片前，自己先编辑好文字，或者，用回车，预留好空白行

- 解决方案2

在编辑界面，点开页面对应的html源代码，在你想插入文字的地方加上html标签 `<p></p>`（图中①位置），然后再回到编辑界面，就会看到空白行。可以在那里编辑。



另，博客园平台设置页面支持更改默认编辑器（图中②位置）。目前，默认编辑器是markdown编辑器。

如果你暂不熟悉markdown基础用法，可切换成TinyMCE编辑器，其界面和使用风格接近office系列。

**markdown编辑器使用参考：** <https://www.runoob.com/markdown/md-tutorial.html>